第七讲教学设计：MVC模式—构建专业的Web应用架构 (20分钟)

课程信息

- **课程名称**: Java Web应用开发 (Java Web Application Development)
- **教学方式**: 双语教学（英文授课，关键术语中英文对照）
- **教学主题**: MVC模式：实现关注点分离
- **授课时长**: 20分钟
- **适用对象**: 已掌握Servlet, JSP, JavaBean, JDBC基础，渴望提升代码架构能力的学生

一、教学目标

知识目标

1. 掌握MVC（Model-View-Controller）设计模式的概念、特点及各部分核心职责。
2. 理解JSP Model 1和Model 2架构的根本区别与演进过程。
3. 掌握使用 `RequestDispatcher` 进行请求转发（Forward）的核心机制。
4. 辨析请求转发（Forward）和重定向（Redirect）的区别与应用场景。

能力目标

1. 能够根据功能需求，识别出业务中对应的M, V, C组件。
2. 能够绘制出MVC模式下一次完整请求的数据流图。
3. 能够将一个简单的功能（如用户登录）按照MVC模式进行代码分层组织。

素质目标

1. 建立"高内聚、低耦合"和"关注点分离"的软件工程核心思想。
2. 培养从"功能实现"到"架构设计"的思维升级。
3. **[思政融合]** 讲解MVC时，类比现代化城市规划（功能分区、交通枢纽、住宅外观），引申到软件工程的模块化、可维护性，这与国家倡导的绿色、集约、可持续发展理念异曲同工，旨在构建高质量、能长远发展的"数字城市"。

二、教学重点与难点

教学重点

- Model, View, Controller三者的职责划分与协同工作的流程。
- JSP Model 2 (Servlet+JSP+JavaBean) 的工作流程。
- 请求转发（`forward`）和重定向（`redirect`）的本质区别。

教学难点

- **概念难点**: 真正理解数据是如何在MVC的三个组件之间流动和共享的。
- **实践难点**: 在没有框架辅助的情况下，如何从零开始手动组织和搭建一个符合MVC规范的项目结构。

三、教学内容与时间安排

第一部分：问题引入 - "厨房里的革命" (4分钟)

内容要点

1. **回顾与痛点 (Recalling the Pain Point):**
   - 展示一幅JSP页面混杂着Java数据库代码的"意大利面条式"代码截图。
   - **设问**: "同学们，回顾我们之前的代码，JSP里既有HTML，又有Java逻辑，甚至还可能直接连数据库。这就像一个餐厅的厨房，厨师（Java逻辑）不仅要炒菜，还要自己跑去前台接单（处理请求），自己端盘子上菜（生成响应），甚至还要自己设计菜单样式（HTML/CSS）。"
2. **场景冲突 (The Chaos):**
   - **生动比喻**: "现在餐厅生意火爆（项目变复杂），问题来了：
     - **效率低下**: 厨师一个人干所有活，手忙脚乱，疲于奔命。
     - **分工混乱**: 接单的服务员、炒菜的厨师、摆盘的艺术家，所有角色混在一起，谁也做不精。
     - **难以维护**: 想换个菜单样式（改页面），结果整个厨房都要停工，连厨师的代码都要小心翼翼地改。想加个新菜（改业务），服务员和摆盘师的流程可能都得跟着变。这是一种灾难！"
3. **引出解决方案 (Introducing the Solution):**
   - **提出革命**: "如何解决这种混乱？我们需要一场'厨房革命'！引入一个科学的管理模式，让'**专业的人做专业的事**'。这个伟大的管理模式，就是我们今天要学习的 **MVC（Model-View-Controller）设计模式**！"

教学方法

- 故事化、场景化教学，用生动的"厨房比喻"制造"痛点"，激发学生对解决方案的渴望。
- 对比法，强化现有模式的弊端，突出MVC的必要性。

第二部分：新知讲解 - MVC的专业分工 (10分钟)

内容要点

1. **MVC三剑客的职责 (The Three Components):**
   - **控制器 (Controller) - 餐厅前台经理**:
     - **职责**: 餐厅的唯一入口，负责接待所有顾客（接收所有HTTP请求）。他不亲自处理，而是根据顾客的点单（请求URL），决定将任务分派给哪个后厨团队。
     - **技术实现**: `Servlet`
   - **模型 (Model) - 后厨核心团队**:
     - **职责**: 负责处理具体的业务。接到经理的指令后，他们或准备食材（调用DAO操作数据库），或烹饪菜品（执行业务逻辑），最后将做好的菜品（封装好的JavaBean）交给经理。
     - **技术实现**: `JavaBean` + 业务逻辑类（`Service`，`DAO`）
     - **代码示例**:

```
// User.java (JavaBean)
public class User { /* ... fields, getters, setters ... */ }
```

```java
// UserService.java (业务逻辑类)
public class UserService {
    public User login(String username, String password) {
        // ... 业务逻辑 ...
        return user;
    }
}
```

- **视图 (View) - 菜品呈现艺术家**:
  - **职责**: 负责将经理递过来的、已经做好的菜品（数据模型），进行精美地摆盘和呈现给顾客。他们只管展示，不关心菜是怎么做的。
  - **技术实现**: `JSP` / `HTML`
  - **代码示例**:

```jsp
// welcome.jsp
<h1>欢迎您, ${userInfo.nickname}</h1>
```

2. **数据流动之旅 (Data Flow Journey):**
   - 用一张清晰的动态图或分步图来展示请求的旅程:
     1. **浏览器** 发出请求，**Controller(Servlet)** 捕获。
     2. **Controller** 调用 **Model(业务逻辑类)** 处理请求。
        - **代码示例 (`LoginServlet.java`):**

```java
UserService userService = new UserService();
User user = userService.login(username, password);
```

     3. **Model** (可能调用DAO) 完成处理，将结果数据返回给 **Controller**。
     4. **Controller** 将数据存入 `request` 作用域，然后请求转发给 **View(JSP)**。
        - **代码示例 (`LoginServlet.java`):**

```java
request.setAttribute("userInfo", user);
request.getRequestDispatcher("/welcome.jsp").forward(request, response);
```

     5. **View** 从 `request` 中取出数据，动态渲染页面，最终生成HTML响应给**浏览器**。
        - **代码示例 (`welcome.jsp`):**

```jsp
欢迎您, ${userInfo.nickname}
```

3. **架构的演进 (Architecture Evolution):**
   - 展示从"**JSP Model 1**" (JSP + JavaBean，即"厨师兼职服务员"模式) 到 "**JSP Model 2**" (Servlet + JSP + JavaBean，即MVC模式) 的架构演进图。
   - 强调Model 2实现了清晰的职责分离，是现代Java Web开发的标准实践。

教学方法

- 延续"厨房比喻"，将抽象概念具体化、角色化。
- 图示法，清晰展示数据流和架构演进，建立宏观认知。

第三部分：核心调度技术 - 转发与重定向 (4分钟)

内容要点

1. **请求转发 (Forward):**
   - **比喻**: **内部转交。**顾客在前台点单，经理直接把单子转给后厨处理，整个过程顾客都在前台等待，他感觉自己从未移动，地址栏URL**不变。**
   - **特点**: 服务器内部行为，仅发生**一次**请求，`request` 对象可以共享。
   - **代码**: `request.getRequestDispatcher("/userView.jsp").forward(request, response);`
2. **重定向 (Redirect):**
   - **比喻**: **请去别处。**顾客在前台办完业务（如付款），经理告诉他："好了，请去大厅休息区等候"，然后顾客自己走到休息区。这是两个独立的动作，地址栏URL**改变。**
   - **特点**: 客户端行为，先后发生**两次**请求，`request` 对象不共享。
   - **代码**: `response.sendRedirect(request.getContextPath() + "/index.jsp");`
3. **场景总结**:
   - **用转发**: **请求需要携带数据到下一个页面进行展示。** 这是最常用的场景。例如，`LoginServlet` 查询到用户信息后，需要将 `User` 对象转发给 `welcome.jsp` 来显示。

     ```
     // LoginServlet.java
     User user = userService.login(...);
     request.setAttribute("user", user);
     request.getRequestDispatcher("/welcome.jsp").forward(request, response);
     ```

   - **用重定向**: **用于避免刷新时表单的重复提交。** 当用户通过POST请求完成一次增、删、改操作后（如提交订单、发表评论），如果直接转发到成功页面，用户刷新浏览器（F5）会导致该POST请求被重新发送，造成数据重复。正确的做法是重定向到一个展示成功信息的GET请求页面。

     ```
     // AddOrderServlet.java
     orderService.addOrder(order); // 数据库操作
     // 不要转发，而是重定向到成功页
     response.sendRedirect(request.getContextPath() + "/addOrderSuccess.jsp");
     ```

教学方法

- 对比教学法，通过生动比喻和表格对比，清晰区分两个概念。

第四部分：课堂总结与展望 (2分钟)

总结要点

1. **MVC是什么?** 一种将应用程序分为模型、视图、控制器三层的架构模式，旨在**实现关注点分离。**
2. **M, V, C分别是谁?** Model(业务+数据), View(JSP), Controller(Servlet)。
3. **MVC如何工作?** Controller统一调度，Model处理业务，View负责显示。

4. **如何调度?** 主要通过请求转发（`forward`）来传递数据和流程。

展望 (承上启下)

- "今天，我们掌握了MVC这座现代化Web应用大厦的设计蓝图。我们知道了每个房间的功能和它们之间的走廊。"
- "下一节课，我们将成为真正的建筑师！我们将亲自动手，把之前学的所有技术（Servlet, JSP, JavaBean, JDBC）作为砖瓦，严格按照MVC这张蓝图，搭建起我们第一个完整的、结构清晰的Web应用——'图书列表展示'。敬请期待！"

四、教学方法与手段

教学方法

1. **故事化比喻法 (Analogy)**: "厨房革命"生动比喻MVC分工
2. **问题驱动法 (Problem-Driven)**: 从代码混乱问题引出MVC
3. **对比分析法 (Comparison)**: Model 1 vs Model 2架构对比
4. **图示法 (Diagram)**: 数据流程图清晰展示
5. **双语教学法 (Bilingual Teaching)**:
    - 课堂采用英文授课，关键技术术语提供中英文对照
    - 重要概念（Model, View, Controller, Forward, Redirect, Separation of Concerns）首次出现时给出英文表达和中文释义
    - 鼓励学生使用英文术语描述MVC架构
    - 代码注释和变量命名使用规范英文

教学手段

- 多媒体课件 (Marp Slides/PPT)
- IDE (IntelliJ IDEA/Eclipse)
- 流程图 (Flow Chart)

五、教学评价

- **过程性评价**: 能否用"厨房比喻"复述M、V、C各自的职责；能否在提问中区分转发和重定向。
- **终结性评价**: 能否独立画出MVC的数据流图；能否将一个简单的业务需求，拆解成M、V、C三个部分。

六、课后拓展

思考题

1. 在MVC模式中，为什么通常使用请求转发而不是重定向来将数据从Controller传递给View?
2. 如果一个功能非常简单，比如只显示一个静态欢迎页面，还有必要严格遵循MVC模式吗？为什么?

实践作业

1. **理论设计**: 请为"用户登录"功能设计一个MVC实现方案。明确指出哪个类是Controller，哪个JSP是View，哪些JavaBean是Model，并画出详细的请求处理流程图。
2. **项目准备**: 在IDE中创建一个新的Web项目，并按照 `controller`，`service`，`dao`，`model` 的结构创建好包（package），并在 `webapp` 下创建 `jsp` 文件夹。为下一节课的综合实战做好准备。

```
/src/com/yourname
    /controller
    /service
    /dao
    /model
/webapp
    /jsp
```